

CENTRO PAULA SOUZA

FACULDADE DE TECNOLOGIA DE FRANCA

“Dr. THOMAZ NOVELINO”

**TECNOLOGIA EM DESENVOLVIMENTO SOFTWARE
MULTIPLATAFORMA**

ANA JÚLIA ALVES MOTA

LAUANE GABRIELA DE ARAÚJO TOLEDO

PEDRO HENRIQUE CINTRA SILVA

DOEFÁCIL - 1ª SPRINT

**Sistema Multiplataforma para Gerenciamento de Doações em
Instituições Sociais**

FRANCA/SP

2026

SUMÁRIO

1- INTRODUÇÃO	1
2- REQUISITOS DO SISTEMA	3
3- MODELAGEM INICIAL	6
4- REPOSITÓRIO E CONTROLE DE VERSÃO	9
5- ESTRUTURA INICIAL DO BACK-END	10
6- PROTÓTIPO INICIAL DO FRONT-END	11
7- MODELAGEM DO BANCO DE DADOS	12
8- COMPUTAÇÃO EM NUVEM II	14
9- MINERAÇÃO DE DADOS	15
10- CONSIDERAÇÕES FINAIS	16
REFERÊNCIAS	17

1. INTRODUÇÃO

O DoeFácil é um sistema multiplataforma voltado ao gerenciamento de doações recebidas por instituições sociais. A proposta surgiu da necessidade de tornar mais organizado o processo de cadastro, recebimento, armazenamento e distribuição de itens como alimentos não perecíveis, roupas, brinquedos e produtos de higiene.

Muitas instituições ainda realizam esse controle por meio de papéis ou planilhas separadas. Essa situação pode dificultar o acompanhamento das campanhas, a consulta do estoque e a identificação dos itens que estão em falta. A solução proposta reúne essas informações em um único sistema, com acesso adequado para doadores, funcionários e administradores.

1.1 Objetivo geral

Desenvolver um sistema multiplataforma que facilite o gerenciamento de doações recebidas por instituições sociais, melhorando o controle das campanhas, das doações e do estoque.

1.2 Objetivos específicos

- Permitir o cadastro e o acompanhamento de doações.
- Organizar campanhas de arrecadação criadas pela instituição.
- Controlar a entrada e a saída dos itens no estoque.
- Disponibilizar informações e relatórios para auxiliar a administração.
- Integrar as aplicações Web, Mobile e Desktop por meio de uma API.
- Utilizar dados históricos para identificar padrões de doação e necessidades de estoque.

1.3 Escopo do projeto

O projeto será desenvolvido inicialmente para uma única instituição social. As aplicações compartilharão a mesma API e o mesmo banco de dados, mantendo as informações centralizadas.

- **Aplicativo mobile:** permitirá que o doador visualize campanhas, cadastre uma doação e acompanhe sua situação.
- **Aplicação web:** será utilizada pelo administrador para gerenciar campanhas, acompanhar doações, consultar o estoque e visualizar relatórios.

- **Aplicação desktop:** será utilizada pelos funcionários para confirmar o recebimento e controlar a entrada e a saída dos itens.

1.4 Público-alvo

- Instituições sociais.
- Funcionários e administradores.
- Pessoas interessadas em realizar doações.

1.5 Funcionalidades incluídas

- Cadastro e login de usuários.
- Cadastro e consulta de campanhas.
- Registro e acompanhamento de doações.
- Confirmação do recebimento.
- Controle de entrada e saída do estoque.
- Consulta de relatórios.
- Envio de notificações e alertas.

1.6 Limites da primeira versão

Para manter o projeto viável dentro do semestre, a primeira versão não incluirá doações em dinheiro, pagamentos, serviço de entrega, chat, mapas, atendimento a várias instituições ou armazenamento de informações sensíveis dos beneficiários.

2. REQUISITOS DO SISTEMA

Os requisitos foram definidos a partir do escopo inicial e representam as funções e características necessárias para o funcionamento do DoeFácil.

2.1 Requisitos funcionais

Quadro 1 – Requisitos funcionais

Código	Descrição
RF01	Permitir o cadastro de usuários.
RF02	Permitir que o usuário realize login.
RF03	Permitir que o doador visualize as campanhas disponíveis.
RF04	Permitir que o doador cadastre uma nova doação.
RF05	Permitir que o doador acompanhe a situação da sua doação.
RF06	Permitir que o administrador cadastre, edite e encerre campanhas.
RF07	Permitir que o funcionário confirme o recebimento das doações.
RF08	Registrar no estoque os itens recebidos.
RF09	Permitir que o funcionário registre a saída dos itens distribuídos.
RF10	Permitir que o administrador consulte o estoque da instituição.
RF11	Gerar relatórios sobre campanhas, doações e estoque.
RF12	Enviar alertas sobre novas doações e itens com estoque baixo.

2.2 Requisitos não funcionais

Quadro 2 – Requisitos não funcionais

Código	Descrição
RNF01	Possuir aplicações Web, Mobile e Desktop.
RNF02	Integrar as aplicações por meio de uma API REST.
RNF03	Utilizar um banco de dados relacional para armazenar as informações.
RNF04	Armazenar as senhas de forma segura, utilizando hash.
RNF05	Exigir autenticação para o acesso às funções administrativas.
RNF06	Apresentar telas simples, organizadas e de fácil utilização.
RNF07	Responder às principais operações em até três segundos, em condições normais de uso.
RNF08	Realizar cópias de segurança periódicas dos dados.
RNF09	Ser hospedado em ambiente de nuvem com possibilidade de aumento de capacidade.
RNF10	Manter o código-fonte versionado utilizando Git e GitHub.

3. MODELAGEM INICIAL

3.1 Atores do sistema

Foram definidos três atores principais para a primeira versão do sistema:

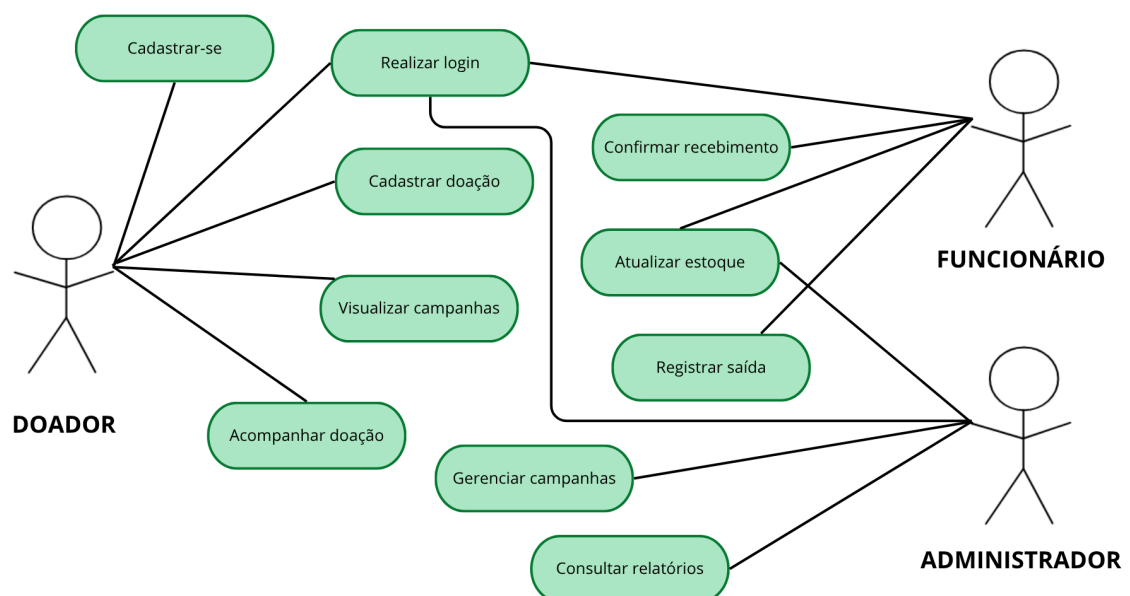
- **Doador:** pessoa que visualiza campanhas, registra doações e acompanha seu andamento.
- **Funcionário:** responsável por confirmar o recebimento e controlar a entrada e a saída dos itens.
- **Administrador:** responsável por gerenciar campanhas, consultar o estoque e visualizar relatórios.

3.2 Casos de uso

Quadro 3 – Casos de uso por ator

Ator	Casos de uso
Doador	Cadastrar-se, realizar login, visualizar campanhas, cadastrar doação e acompanhar doação.
Funcionário	Realizar login, confirmar recebimento, atualizar estoque e registrar saída de itens.
Administrador	Realizar login, gerenciar campanhas, consultar estoque e consultar relatórios.

Figura 1 – Diagrama de casos de uso do DoeFácil

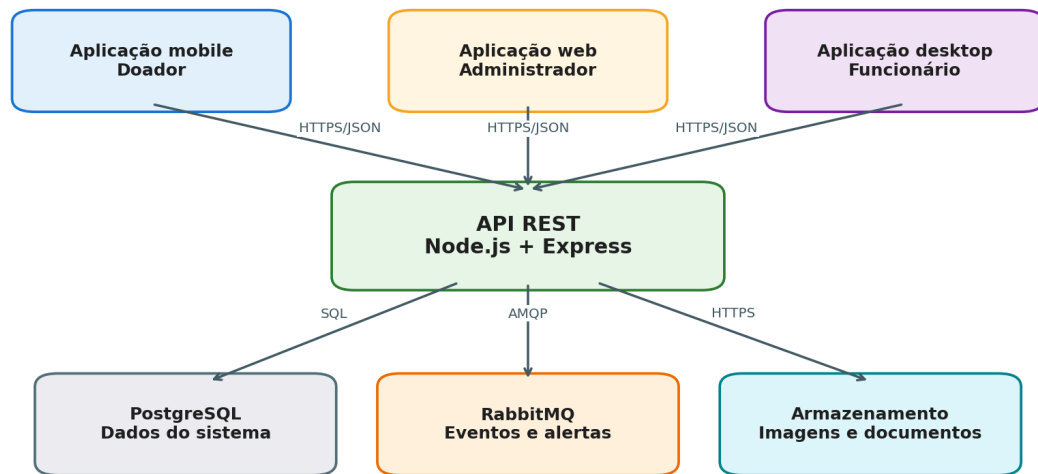


3.3 Arquitetura do sistema

A arquitetura adotará uma API REST central para receber as requisições das aplicações Mobile, Web e Desktop. A API concentrará as regras de negócio e fará a comunicação com o banco de dados, o serviço de mensageria e o armazenamento de arquivos.

Figura 2 – Arquitetura inicial do sistema

Arquitetura inicial do DoeFácil



Todas as aplicações consomem a mesma API e compartilham as regras de negócio.

4. REPOSITÓRIO E CONTROLE DE VERSÃO

O código-fonte será mantido em um repositório compartilhado no GitHub. Cada integrante trabalhará em uma ramificação própria e as alterações serão integradas à ramificação principal após revisão do grupo.

4.1 Identificação do repositório

Quadro 4 – Informações do repositório

Item	Definição
Nome	DSM-G06-PI6-2026-2
Endereço	https://github.com/FatecFranca/DSM-G06-PI6-2026-2
Ramificação principal	main
Documentação	README.md e pasta docs

4.2 Estrutura inicial

Quadro 5 – Organização das pastas

Pasta	Conteúdo
backend/	API, regras de negócio e testes.
frontend-web/	Aplicação Web administrativa.
mobile/	Aplicativo destinado aos doadores.
desktop/	Aplicação de controle interno e estoque.
database/	Modelo, scripts e dados iniciais.
docs/	Requisitos, diagramas e documentos da Sprint.

4.3 Fluxo de versionamento

1. Atualizar a ramificação principal.
2. Criar uma ramificação para a atividade.
3. Realizar alterações e commits objetivos.
4. Enviar a ramificação ao GitHub.
5. Abrir uma solicitação de integração e revisar antes de unir à main.

5. ESTRUTURA INICIAL DO BACK-END

Para a estrutura inicial do back-end, foi selecionado o Node.js com o framework Express e a linguagem TypeScript. Essa combinação permite criar a API REST de maneira organizada, com separação entre rotas, controladores, serviços e acesso ao banco de dados.

5.1 Tecnologias previstas

Quadro 6 – Tecnologias do back-end

Tecnologia	Finalidade
Node.js	Ambiente de execução do back-end.
Express	Framework para criação da API REST.
TypeScript	Tipagem e melhor organização do código.
Prisma	Mapeamento e acesso ao banco PostgreSQL.
JWT	Autenticação dos usuários.
Jest	Execução de testes automatizados.

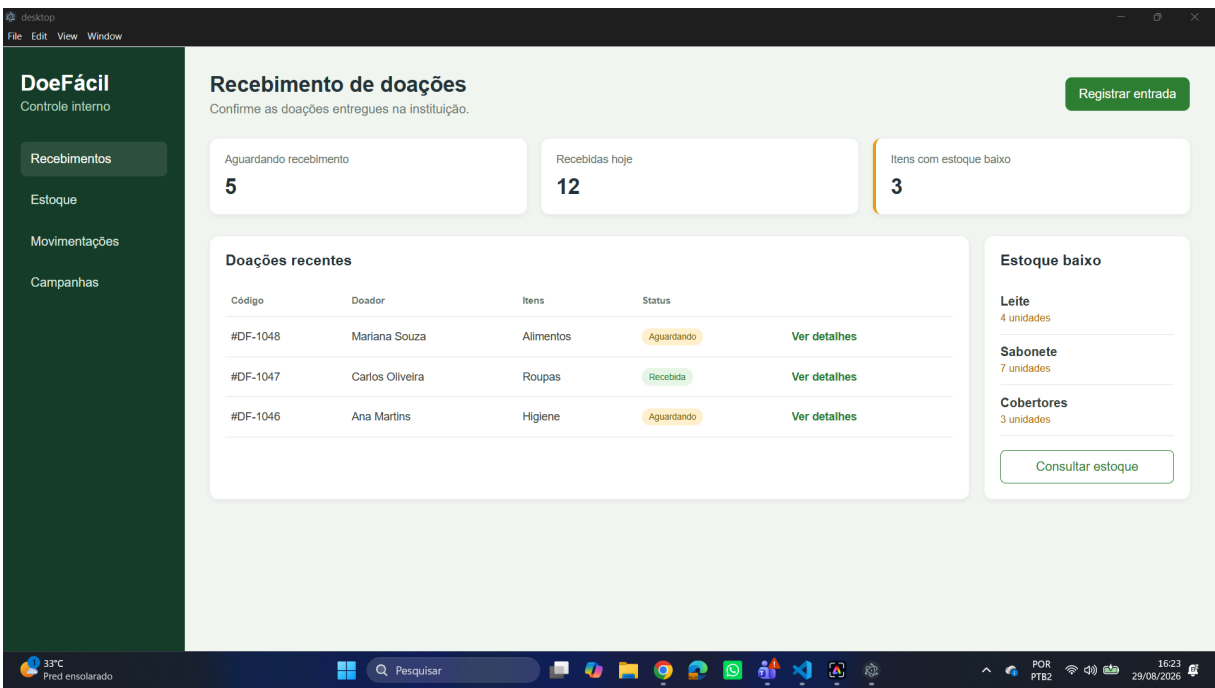
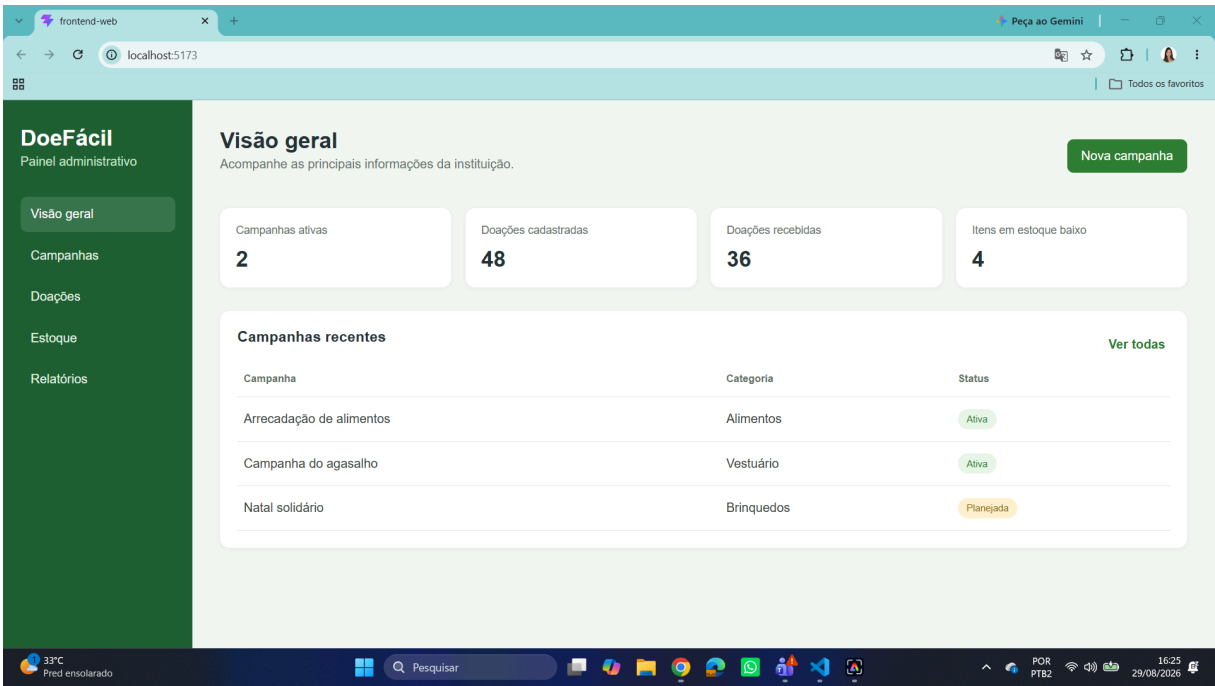
5.2 Rotas iniciais da API

Quadro 7 – Rotas planejadas para a API

Método	Rota	Finalidade
GET	/api/health	Verificar se a API está funcionando.
POST	/api/auth/cadastro	Cadastrar um usuário.
POST	/api/auth/login	Autenticar um usuário.
GET	/api/campanhas	Listar campanhas.
POST	/api/doacoes	Registrar uma doação.
PATCH	/api/doacoes/:id/status	Atualizar o status de uma doação.
GET	/api/estoque	Consultar o estoque.
POST	/api/movimentacoes	Registrar entrada ou saída de item.

6. PROTÓTIPO INICIAL DO FRONT-END

Os protótipos estáticos foram elaborados para representar a organização inicial das principais telas. Nesta Sprint, o objetivo é validar a navegação e a distribuição das informações antes da implementação completa das funcionalidades.



16:04

55

DoeFácil

Sua doação pode transformar vidas.

Campanhas disponíveis

ALIMENTOS

Arrecadação de alimentos

Ajude com alimentos não perecíveis para as famílias atendidas.

Quero doar

VESTUÁRIO

Campanha do agasalho

Estamos recebendo roupas e cobertores em bom estado.

Quero doar

Acompanhar minhas doações

Quadro 8 – Telas previstas por plataforma

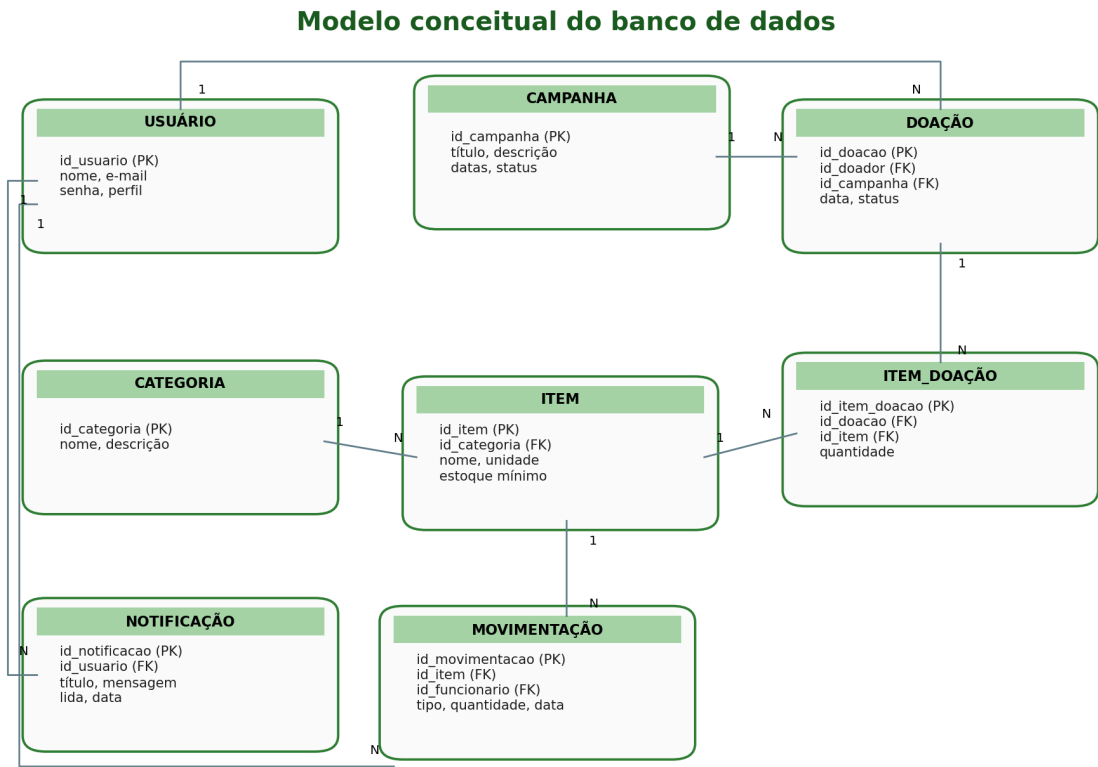
Plataforma	Telas iniciais
Mobile	Campanhas disponíveis; cadastro e acompanhamento de doações.
Web	Painel administrativo; campanhas; relatórios e visão do estoque.
Desktop	Recebimento de doações; entrada e saída de itens do estoque.

7. MODELAGEM DO BANCO DE DADOS

O banco de dados será relacional e utilizará o PostgreSQL. O modelo foi organizado para registrar usuários, campanhas, doações, itens, categorias, movimentações de estoque e notificações.

7.1 Modelo conceitual

Figura 4 – Modelo conceitual do banco de dados



7.2 Modelo lógico

Quadro 9 – Estrutura lógica das tabelas

Tabela	Chave primária	Principais campos
USUARIO	id_usuario	nome, email, senha_hash, perfil, data_cadastro
CAMPANHA	id_campanha	titulo, descricao, data_inicio, data_fim, status
CATEGORIA	id_categoria	nome, descricao
ITEM	id_item	id_categoria, nome, unidade_medida, estoque_minimo
DOACAO	id_doacao	id_doador, id_campanha, data_cadastro, status, observacao
ITEM_DOACAO	id_item_doacao	id_doacao, id_item, quantidade

Tabela	Chave primária	Principais campos
MOVIMENTACAO_ESTOQUE	id_movimentacao	id_item, id_funcionario, id_doacao, tipo, quantidade, data
NOTIFICACAO	id_notificacao	id_usuario, titulo, mensagem, lida, data_criacao

7.3 Regras de integridade

- O e-mail do usuário deverá ser único.
- A quantidade registrada em uma doação deverá ser maior que zero.
- Uma movimentação deverá ser identificada como entrada ou saída.
- A saída de um item não poderá deixar o estoque com quantidade negativa.
- Campanhas e doações utilizarão estados previamente definidos pelo sistema.

8. COMPUTAÇÃO EM NUVEM II

Para a futura implantação, foi escolhida a Amazon Web Services (AWS). A utilização de serviços gerenciados reduz a necessidade de manter servidores físicos e permite ajustar os recursos de acordo com o crescimento do sistema.

Quadro 10 – Serviços em nuvem selecionados

Serviço	Utilização	Justificativa
Elastic Beanstalk	Hospedagem e gerenciamento da API Node.js.	Facilita a implantação e permite escalabilidade com balanceamento de carga.
Amazon RDS for PostgreSQL	Banco de dados relacional gerenciado.	Oferece backups, monitoramento e possibilidade de alta disponibilidade.
Amazon S3	Armazenamento de imagens e arquivos.	Possui alta durabilidade e acesso controlado.
Amazon MQ for RabbitMQ	Mensageria entre os serviços.	Processa notificações e eventos sem bloquear as operações principais.
Amazon CloudWatch	Monitoramento e registros.	Permite acompanhar falhas, desempenho e uso dos recursos.

8.1 Segurança, disponibilidade e custos

O acesso aos serviços será controlado por permissões específicas. A comunicação externa utilizará HTTPS, o banco ficará em rede privada e as credenciais serão armazenadas em variáveis de ambiente. Para disponibilidade, serão utilizados backups automáticos, monitoramento e, conforme a necessidade, mais de uma instância da API. Durante o desenvolvimento serão priorizadas faixas gratuitas e configurações de menor custo.

8.2 Uso da mensageria

O RabbitMQ será utilizado para publicar eventos como doação cadastrada, doação recebida e estoque abaixo do mínimo. Consumidores separados poderão processar notificações e relatórios de forma assíncrona, evitando que essas tarefas aumentem o tempo de resposta da API.

9. MINERAÇÃO DE DADOS

A base utilizada na mineração será formada pelos registros de campanhas, doações, itens e movimentações de estoque. Como o sistema ainda não possui histórico real, os primeiros testes serão realizados com dados simulados, mantendo a mesma estrutura prevista para a base definitiva.

9.1 Dados selecionados

Quadro 11 – Grupos de dados para mineração

Origem	Atributos principais
Doação	Data, campanha, situação e quantidade de itens.
Item	Categoria, unidade, estoque mínimo e estoque atual.
Movimentação	Tipo, quantidade e data da entrada ou saída.
Campanha	Período, situação e tipos de itens solicitados.

9.2 Técnicas planejadas

Quadro 12 – Técnicas de mineração planejadas

Técnica	Aplicação no DoeFácil
Análise descritiva	Identificar itens mais doados, itens mais distribuídos e períodos de maior arrecadação.
Aprendizado supervisionado	Prever a possibilidade de um item atingir estoque baixo a partir do histórico de entradas e saídas.
Aprendizado não supervisionado	Agrupar campanhas ou períodos com padrões semelhantes por meio do algoritmo K-Means.

9.3 Etapas iniciais

1. Compreender o problema e definir as perguntas que serão respondidas.
2. Selecionar e organizar os dados necessários.
3. Tratar valores ausentes, duplicidades e inconsistências.
4. Analisar os dados e criar visualizações iniciais.
5. Treinar e avaliar os modelos selecionados.
6. Apresentar os resultados no painel administrativo.

10. CONSIDERAÇÕES FINAIS

A primeira Sprint definiu a base do DoeFácil, incluindo o escopo, os requisitos, os atores, a arquitetura, o modelo de dados, os serviços em nuvem e o planejamento da mineração de dados. Também foram estabelecidas a organização do repositório, a estrutura inicial da API e as principais telas das três plataformas.

Nas próximas etapas, o grupo deverá implementar as funcionalidades planejadas, conectar as aplicações à API, criar o banco de dados, configurar a mensageria, ampliar os testes e preparar a implantação em nuvem. O escopo reduzido permitirá que a equipe entregue uma solução funcional sem perder a possibilidade de expansão futura.

REFERÊNCIAS

AMAZON WEB SERVICES. AWS Elastic Beanstalk Documentation. Disponível em: <https://docs.aws.amazon.com/elasticbeanstalk/>. Acesso em: 29 ago. 2026.

AMAZON WEB SERVICES. Amazon RDS Documentation. Disponível em: <https://docs.aws.amazon.com/rds/>. Acesso em: 29 ago. 2026.

EXPRESS.JS. Express documentation. Disponível em: <https://expressjs.com/>. Acesso em: 29 ago. 2026.

NODE.JS. Node.js documentation. Disponível em: <https://nodejs.org/docs/>. Acesso em: 29 ago. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL documentation. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 29 ago. 2026.

RABBITMQ. RabbitMQ documentation. Disponível em: <https://www.rabbitmq.com/docs>. Acesso em: 29 ago. 2026.